

Дисциплина «Искусственный интеллект»

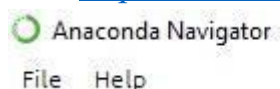
Рабочая тетрадь № 1

Цель изучения дисциплины – формирование компетенций в области современных информационных технологий с применением инструментария искусственного интеллекта и машинного обучения. Для реализации методов и алгоритмов будем использовать язык программирования Python.

Python является популярным высокоуровневым языком программирования, который применяется для решения прикладных задач принятия решений с использованием технологий искусственного интеллекта и машинного обучения.

Для реализации методов и алгоритмов, изучаемых в данной дисциплине, рекомендуется использовать среду разработок Anaconda, содержащую библиотеки языка программирования Python с открытым исходным кодом.

<https://www.anaconda.com/products/individual>

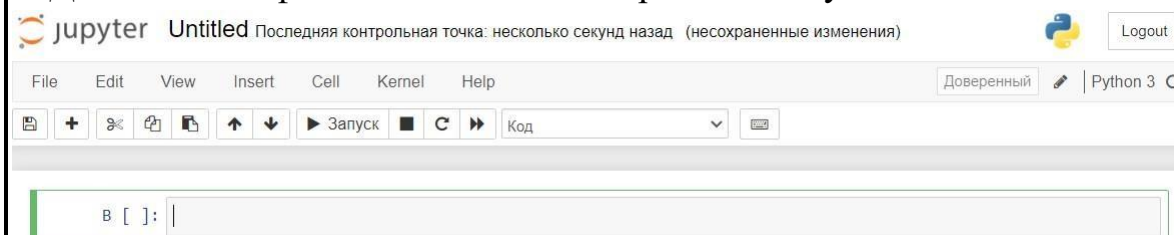


Для выполнения заданий из рабочих тетрадей рекомендуется использовать Jupyter Notebook.



Название рабочих тетрадей должно содержать фамилию и имя студента, номер рабочей тетради. Например, Shmeleva_A_Workbook1.ipynb

Для создания файла необходимо выбрать вкладку **New/Notebook**:



Далее **File/Rename** и указать название файла.

1.1. Теоретический материал – Типы данных

Типы данных

Все типы данных в Python относятся к одной из 2-х категорий: изменяемые (mutable) и неизменяемые (immutable).

Неизменяемые объекты:

- числовые данные (int, float),
- bool,
- None,
- символьные строки (class 'str'),
- кортежи (tuple).

Изменяемые объекты:

- списки (list),
- множества (set),
- словари (dict).

1.2. Пример

Задача:

Выведите на печать и определите тип переменной.

```
x = 3+5.2*7
y = None
z = 'a', 5, 12.345, (2, 'b')
df = [['Антонова Антонина', 34, 'ж'], ['Борисов Борис', 26, 'м']]
A = {1, 'title', 2, 'content'}
```

Решение:

```
print(x, '|', type(x), '\n', y, '|', type(y), '\n',
      z, '|', type(z), '\n', df, '|', type(df), '\n', A, '|', type(A))
```

Ответ:

```
39.4 | <class 'float'>
None | <class 'NoneType'>
('a', 5, 12.345, (2, 'b')) | <class 'tuple'>
[['Антонова Антонина', 34, 'ж'], ['Борисов Борис', 26, 'м']] | <class 'list'>
{1, 2, 'content', 'title'} | <class 'set'>
```

1.3. Задание

1. Задача:

Выведите на печать и определите тип переменной.

```
x = 5 >= 2
A = {1, 3, 7, 8}
B = {2, 4, 5, 10, 'apple'}
C = A & B
df = 'Антонова Антонина', 34, 'ж'
z = 'type'
D = [1, 'title', 2, 'content']
```

Решение:

```
x = 5 >= 2
A = {1,3,7,8}
B = {2,4,5,10,'apple'}
C = A & B
df = 'Антонова Антонина', 34, 'ж'
z = 'type'
D = [1, 'title', 2, 'content']
for value in [x,A,B,C,df,z,D]:
    print(f'{value}: {type(value)}')
```

Ответ:

```
True: <class 'bool'>
{8, 1, 3, 7}: <class 'set'>
{2, 4, 5, 'apple', 10}: <class 'set'>
set(): <class 'set'>
('Антонова Антонина', 34, 'ж'): <class 'tuple'>
type: <class 'str'>
[1, 'title', 2, 'content']: <class 'list'>
```

2.1. Теоретический материал – Условный оператор**If – Условный оператор**

В коде часто приходится проверять выполнимость или невыполнимость каких-то условий. Синтаксис:

```
if <условие1 (булевское выражение)> :
    <код, который выполнится, если условие верно>
elif <условие2 (булевское выражение)>:
    <код, который выполнится, если условие1 было неверно, а
    условие2 верно>
else:
    <код, который выполнится, если условие1 и условие2 были
    неверны>
```

Обратите внимание, что код, который должен выполняться внутри каждого условия, записывается с отступом в 4 пробела от уровня if, elif и else: в питоне области видимости переменных обозначаются отступами.

То есть, отступы позволяют понять, где начинается код, который должен выполняться при выполнении условия в if, и где заканчивается.

2.2. Пример**1. Задача:**

Вывести на экран является ли переменная x положительной, отрицательной или равна нулю.

Решение:

	<pre> x = 125 if x<0: print("x отрицательный") elif x==0: print("x равен нулю") else: print("x положительный") </pre>	
	Ответ:	
	x положительный	
2.3. Задание		
1.	Задача:	
	Напишите код. задается x, напечатать какому из интервалов принадлежит: $(-\infty, -5)$, $[-5, 5]$ или от $(5, +\infty)$	
	Решение:	
	<pre> x = int(input()) if x < -5: print("(-infinity, -5)") elif -5 <= x <= 5: print("[-5;5]") else: print("(5;+infinity)") </pre>	
	Ответ:	
	6 $(5;+\infty)$	

3.1. Теоретический материал – Циклы, массивы, срезы

Циклы - **for** и **while**

Цикл **while** проверяет истинность условия, и если условие истинно, то выполняет инструкции цикла.

Синтаксис:

```
while условие:  
    инструкции
```

Инструкции, которые относятся к циклу **while**, располагаются на последующих строках и должны иметь отступ от начала ключевого слова **while**.

Цикл **for** проходит по набору значений, помещает каждое значение в переменную с которой в цикле можно производить различные действия.

Синтаксис:

```
for переменная in набор_значений:  
    инструкции
```

После ключевого слова **for** идет название переменной, в которую будут помещаться значения. После оператора **in** указывается набор значений и двоеточие. Инструкции, которые относятся к циклу, располагаются на последующих строках и должны иметь отступ.

При создании цикла удобно пользоваться функцией **range(a, b)**, которая создает последовательность чисел от **a** до **b-1**.

3.2.1. Пример

Задача:

Вывести все числа от 1 до 10 с шагом 3.

Решение:

```
x = 1  
while x <= 10:  
    print(x)  
    x += 3
```

Ответ:

```
1  
4  
7  
10
```

3.2.2. Пример

Задача:

Задан список. Напечатать все элементы списка.

Решение:

```
models = ['KNN', 'decision tree', 'linear model']  
for model in models:  
    print(model)
```

Ответ:

KNN
decision tree
linear model

3.2.3. Пример

Задача:

Создать список от 1 до 99 с шагом 7.

Решение:

```
list_int = range(1, 100, 7)
print(list(list_int))
```

Ответ:

```
[1, 8, 15, 22, 29, 36, 43, 50, 57, 64, 71, 78, 85, 92, 99]
```

3.2.4. Пример

Задача:

Вывести на экран числа от 1 до 105 с шагом 25, используя функцию range()

Решение:

```
for i in range(5, 106, 25):
    print(i)
```

Ответ:

```
5
30
55
80
105
```

3.2.5. Пример

Задача:

Срез. Необходимо получить каждый k-й элемент массива a, начиная с элемента с индексом i (включительно) и заканчивая элементом с индексом j (не включительно), с шагом 2. Или переставить все элементы в обратном порядке.

Решение:

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
b = a[1:7:2]
c = a[::-1]
print(b)
print(c)
```

Ответ:

```
[2, 4, 6]
[9, 8, 7, 6, 5, 4, 3, 2, 1]
```

3.3.1 Задание

1.	Задача: Выведите числа из примера на while (3.2.1) в обратном порядке. Решение: <pre># 3.3.1 x = 10 while x > 0: print(x) x -= 3</pre> Ответ: 10 7 4 1
3.3.2 Задание	
1.	Задача: При решении задач машинного обучения часто в качестве объектов исследования выступает человек. Создайте список значимых характеристик (признаков), идентифицирующих человека. Выведите список на экран. Решение: <pre># 3.3.2 l = ['наличие рук', 'уровень когнитивных способностей', 'конкретная форма черепа'] print(l)</pre> Ответ: ['наличие рук', 'уровень когнитивных способностей', 'конкретная форма черепа']
3.3.3 Задание	
1.	Задача: Создать список чисел от 2 до 15 с шагом 1. Решение: <pre># 3.3.3 print([i for i in range(2,15,1)])</pre> Ответ: [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]

3.3.4 Задание

1. **Задача:**

Выведите числа из примера на for с функцией range() (3.2.4) в обратном порядке.

Решение:

```
# 3.3.4  
  
for i in range(105,4,-25):  
    print(i)
```

Ответ:

```
105  
80  
55  
30  
5
```

3.3.5 Задание

1. **Задача:**

Срез. Напишите код, который все элементы массива x с четными индексами переставит в обратном порядке. Т.е. если x = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9], то код должен сформировать [8, 1, 6, 3, 4, 5, 2, 7, 0, 9].

Решение:

```
# 3.3.5  
  
x = list(range(0,10))  
even = x[::2]  
odd = list(reversed(x[1::2]))  
y = []  
l = min(len(even),len(odd))  
  
for i in range(l):  
    y.append(even.pop())  
    y.append(odd.pop())  
  
y = y + odd + even  
print(y)
```

Ответ:

```
[8, 1, 6, 3, 4, 5, 2, 7, 0, 9]
```


4.1. Теоретический материал - Библиотеки

Машинное обучение опирается на фундаментальные понятия и законы математики и информатики. Библиотека математических функций (math) применяется при решении большинства прикладных задач анализа данных. Также, и некоторые другие библиотеки Python содержат математические функции (например, NumPy). Важную роль играет библиотека matplotlib, которая позволяет производить визуализацию расчетов в удобной для восприятия человеком форме: графики, диаграммы, гистограммы.

Подключение библиотек – синтаксис:

```
import math as m
import numpy as np
import matplotlib.pyplot as plt
etc...
```

Библиотеки в python содержат не только функции, позволяющие решать различные задачи, но и некоторые популярные константы, например число Эйлера.

4.2.1 Пример

Задача:

Рассчитать синус от e радиан и вывести результат на экран

Решение:

```
1 import math as m
2 print (m.sin (m.e))
```

Ответ:

0.41078129050290885

4.2.2 Пример

Задача:

Для функции $\sin(xe^{\cos(x)})$ на интервале $[0;10]$ с шагом 0,05 построить график функции, закрасить площадь под ним и вычислить значение этой площади.

Для вычисления площади воспользуйтесь функции `trapz()`, вычисляющей интеграл по правилу трапеции. Для ее корректной работы необходимо подключить следующие библиотеки:

```
from scipy.integrate import.simps
from numpy import trapz
```

Решение:

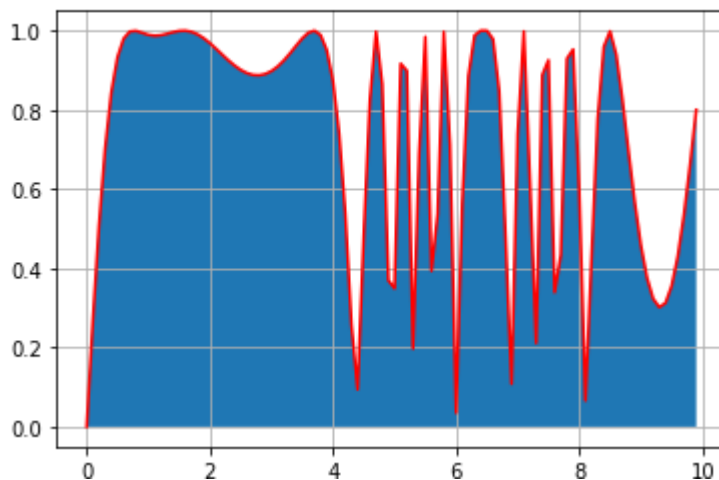
```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import simpson
4 from numpy import trapz
5 x = np.arange(0.0, 10, 0.1)
6 y = np.abs(np.sin(x*np.exp(np.cos(x))))
7 plt.grid()
8 plt.plot(x, y, c = "r")
9 plt.fill_between(x, y)
10
11 area = trapz(y)
12 print (area)

```

Ответ:

73.50792907748662



4.2.3 Пример

Задача:

Дано некоторое распределение оценок в группе за экзамен. Нарисовать круговую и точечную диаграммы распределения.

Решение:

```

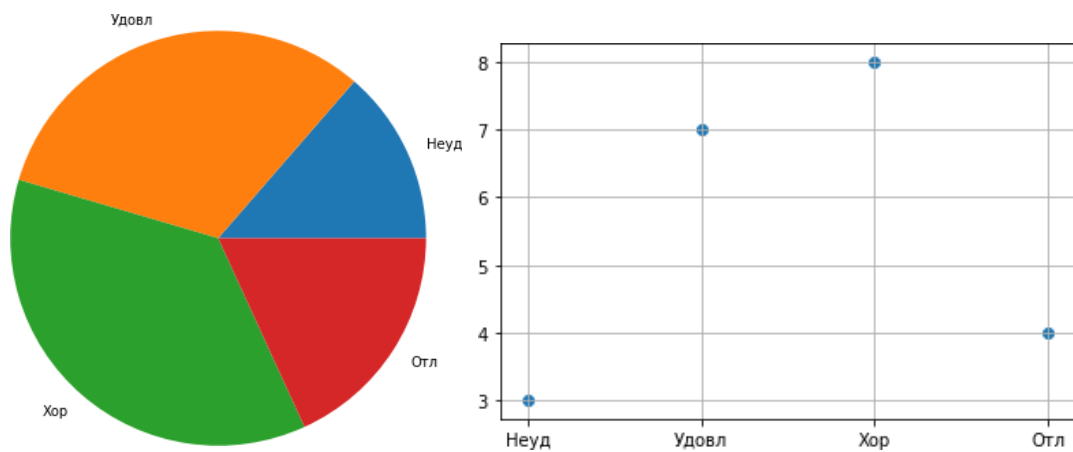
from matplotlib import pyplot as plt
import numpy as np

marks = ['Неуд', 'Удовл', 'Хор', 'Отл']

data = [3, 7, 8, 4]
fig = plt.figure(figsize =(10, 7))
plt.pie(data, labels = marks)
plt.show()
plt.grid()
plt.scatter (marks,data)

```

Ответ:



4.3.1 Задание

1. Задача:

Задайте массив случайных значений из интервала(0; 1).
 Рассчитайте средние и медианные значения для массива, сравните результаты, какие выводы можно сделать о значениях?
 Постройте точечную диаграмму рассеяния полученного ряда.

Решение:

4.3.1

```
import numpy as np
import matplotlib.pyplot as plt
distr = np.random.uniform(size=100)
avg = np.mean(distr)
med = np.median(distr)

# медиана и ср. арифм. примерно одинаковы, если
# данные сгенерированы случайно
print(avg, med)

fig = plt.figure()
plt.show()
plt.grid()
plt.title("Диаграмма рассеяния")
plt.ylabel("Индекс элемента")
plt.xlabel("Значение")
plt.plot([avg, med], [50, 50], marker="o", markerfacecolor="red")
plt.scatter(distr, range(len(distr)))
```

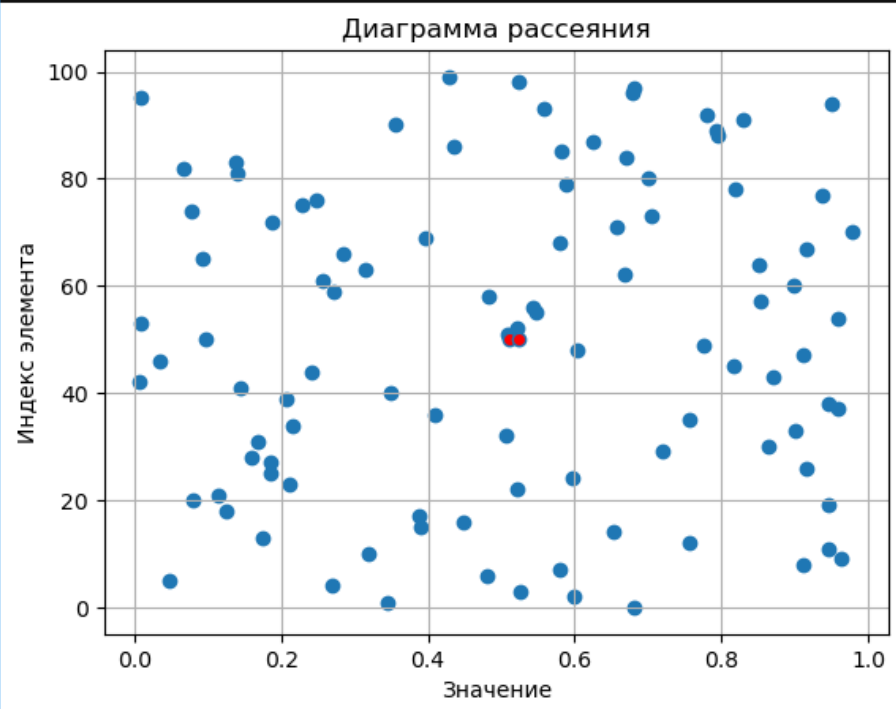
Ответ:

```
0.5114700103844598 0.5242078076983446
```

```
<Figure size 640x480 with 0 Axes>
```

```
[33]:
```

```
<matplotlib.collections.PathCollection at 0x1eec96eafd0>
```



4.3.2 Задание

1. **Задача:**

Дана функция $\frac{\sqrt{1 + e^{\sqrt{x}} + \cos x^2}}{|1 - \sin^3 x|} + \ln|2x|$; Создать массив из 10 значений функции (x , например, изменяется от 1 до 10). Выделить срез первой половины массива и построить графики для основного массива – линейный и для среза – точечный

Решение:

```
# 4.3.2
import numpy as np
import matplotlib.pyplot as plt

def f(values):
    res = []
    for x in values:
        a = np.sqrt(1+np.exp(np.sqrt(x))+np.cos(x**2))
        b = np.abs(1-np.sin(x)**3)
        s = np.log(np.abs(2*x))
        res.append(a/b + s)
    return res

initial = np.arange(1,10)
out = f(initial)

fig = plt.figure()
plt.show()
```

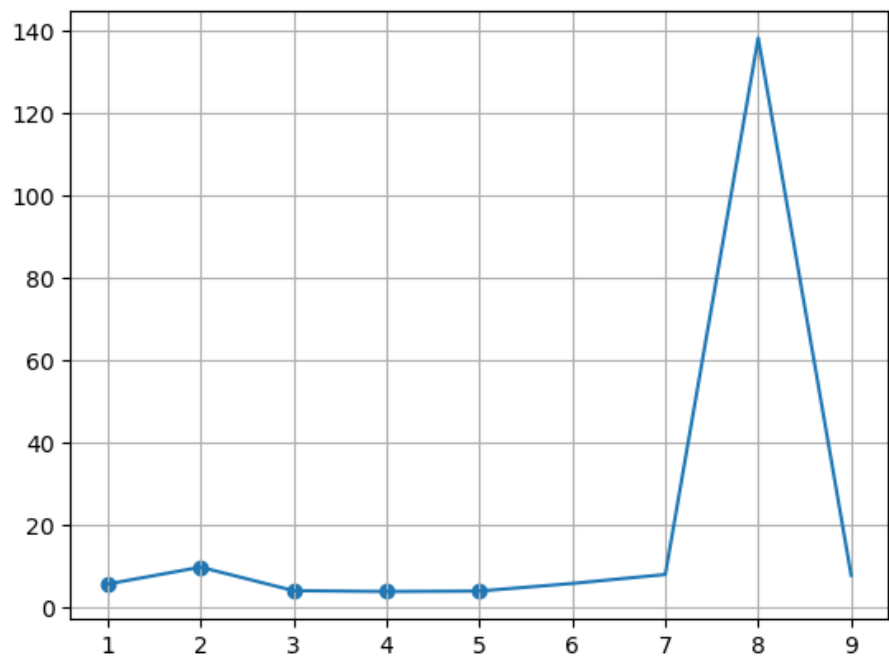
```
plt.grid()
plt.plot(initial,out)
plt.scatter(initial[:5],out[:5])
```

Ответ:

```
<Figure size 640x480 with 0 Axes>
```

```
[6]:
```

```
<matplotlib.collections.PathCollection at 0x2057a2fbed0>
```



4.3.3 Задание

1. **Задача:**

Дана функция $|\cos(xe^{\cos(x)+\ln(x+1)})|$. Построить график на интервале (0,10) с шагом 1 с заливкой площади и найти эту площадь под ним. Для вычисления площади воспользуйтесь функции `trapz()`, вычисляющей интеграл по правилу трапеции.

Для ее корректной работы необходимо подключить следующие библиотеки:

```
from scipy.integrate import simpson
from numpy import trapz
```

Решение:

```
# 4.3.3
```

```
def f(arr):
    sub = np.cos(arr)+np.log(arr+1)
    return np.abs(np.cos(arr*np.e**sub))
```

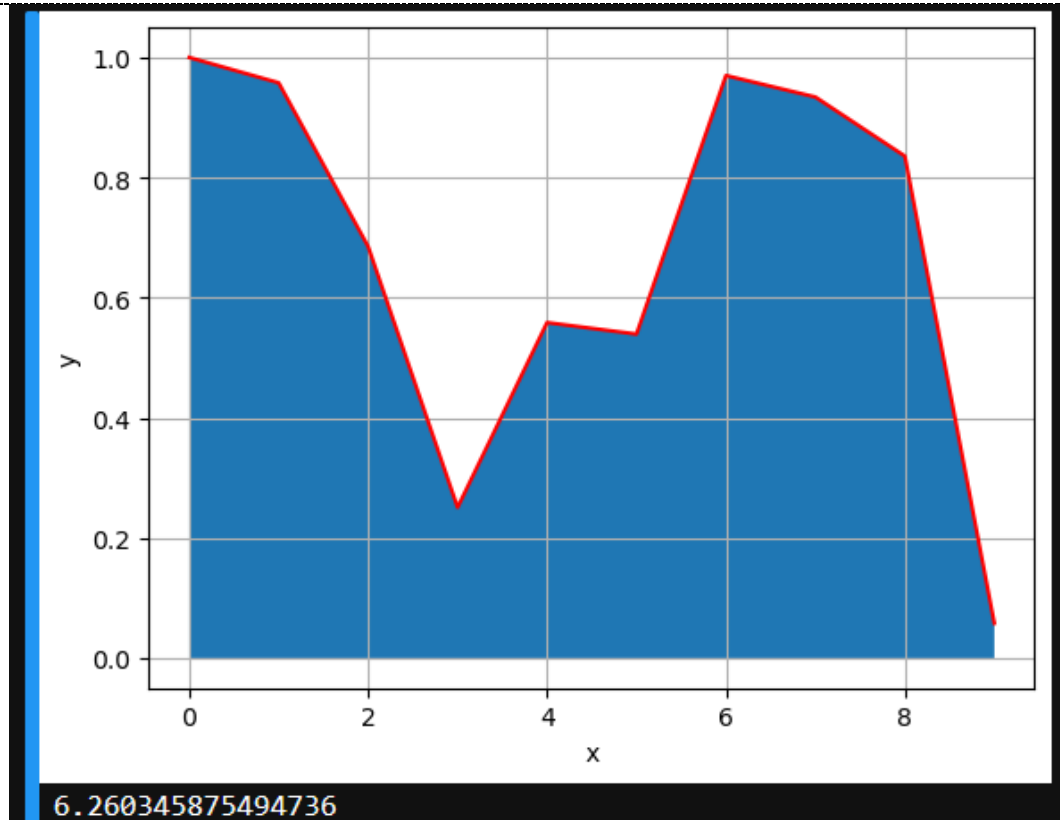
```
x = np.arange(0,10,1)
y = f(x)
```

```
fig = plt.figure()
plt.axes(xlabel="x",ylabel="y")
```

```
plt.plot(x,y,c="r")
plt.fill_between(x,y)
plt.grid()
plt.show()
```

```
S = np.trapezoid(y)
print(S)
```

Ответ:



4.3.4 Задание

1. **Задача:**

Откройте статистические данные по стоимости акций Apple, Microsoft, Google за 12 месяцев 2021 г. Постройте 3 графика на плоскости и оцените их динамику. Подумайте, как можно улучшить читаемость результатов.

Для начальных данных допустимо использовать следующий ресурс:
<https://finance.yahoo.com>

Решение:

```
# 4.3.4
from sklearn.preprocessing import MinMaxScaler
```

```
google = [
    140, 140.84, 139.21, 141.56, 142.85, 144.25, 144.11, 143, 142.2,
    141.65, 138.55, 139.7, 140.76, 140.32, 140.15, 137.85, 137.36,
    135.7, 137.07, 135.59, 137.4, 139.61, 142.5, 142.9, 142.25,
    141.58, 140.81, 139.36, 144.25, 145.2, 145.58, 144.73, 144.19,
    146.02, 145.67, 145.55, 145.82, 145.38, 146.32, 146.82, 146.11,
    146.49, 145.01, 143.13, 143.31, 143.01, 142.18, 138.61, 137.45,
    138.29, 138.72, 138.99, 138.67, 138.42, 138.85, 138.55, 138.32,
    137.08, 136.95, 136.54, 136.34, 136.02, 135.77, 137.15, 139.68,
```

140.01, 139.71, 138.81, 133.5, 132.62, 132, 131.25, 132.18,
132.6, 133, 132.04, 130.77, 129.85, 130.03, 130.64, 129.88,
128.85, 126.46, 125.8, 127, 127.34, 127.5, 127.54, 127.8,
127.27, 127.04, 126.39, 127.2, 126.52, 126.86, 126.41, 126.35,
126.16, 125.25, 124.72, 123.4, 122.69, 120.49, 122.1, 121.9,
121.41, 122, 122.15, 121.64, 120.92, 118.45, 118.02, 115.84,
117.16, 116.17, 116.06, 113.83, 114.27, 116.1, 118.9, 120.82,
119.14, 119.11, 118.96, 120.99, 121.36, 121.83, 122.62, 116.87,
117.06, 116.29, 115.19, 114.77, 115.48, 115.92, 115.32, 115.33,
113.9, 113.86, 113.77, 114.45, 114.2, 112.75, 111.88, 111.87,
107.15, 104.67, 103.54, 102.92, 102.55, 102.94, 103.91, 103.62,
102.9, 102.65, 103.78, 105.49, 106.18, 103.35, 104.51, 106.29,
103.75, 103.9, 106.44, 105.91, 104.46, 104.43, 105.22, 104.33,
103.55, 104.74, 105.04, 104.1, 104.57, 106.53, 106.64, 106.68,
107.63, 105.44, 105.1, 105.42, 105.26, 106.18, 105.13, 103.93,
105.82, 97.79, 96.12, 92.86, 94.94, 94.5, 96.25, 96.48, 95.55,
96.74, 95.19, 90.46, 87.8, 88.75, 88.25, 88.9, 89.72, 90.49,
89.42, 87.4, 87.38, 88.03

]

apple = [

149.37, 150.18, 149.64, 149.75, 149.17, 146.84, 144.9, 143.88, 141.4,
143.25, 144.81, 144.18, 144.22, 142.15, 142.24, 142.21, 142.92,
144.38, 144.45, 144.75, 145.96, 147.47, 147.08, 146.43, 144.6,
144.84, 148.82, 148.97, 149.44, 151.07, 151.42, 155.48, 156.11,
157.04, 157.26, 154.63, 154.72, 154.98, 152.8, 153.49, 148.75,
149.12, 150.32, 150.86, 150.19, 148.5, 148, 150.72, 151.68,
151.19, 149.44, 149.05, 146.72, 147.71, 146.7, 147.11, 147.84,
147.79, 148.04, 146.95, 146.33, 146.55, 146.97, 149.21, 149.83,
148.72, 148.2, 146.13, 147.1, 144.07, 149.76, 150, 149.57,
147.46, 146.32, 145.65, 144.06, 144.89, 143.15, 140, 137.33,
137.41, 136.49, 135.25, 133.89, 134.64, 134.32, 134.08, 132.41,
131.51, 132.55, 130.89, 130.6, 130.54, 127.44, 128.19, 127.75,
128.46, 126.32, 126.16, 124.85, 125.24, 125.35, 125.8, 127.64,
127.39, 128.32, 127.94, 128, 127.72, 124.92, 126.99, 126.93,
127.89, 126.15, 124.64, 126.27, 129.54, 131.26, 129.75, 130.45,
131.49, 134.07, 133.56, 137.07, 135.02, 135.41, 135.06, 135.12,
134.15, 133.75, 135.53, 135.47, 134.67, 135, 135, 134.66,
132.85, 133.04, 130.39, 127.92, 127.13, 126.16, 124.18, 123.52,
120.4, 122.58, 121.48, 121.66, 122.9, 124.24, 123.87, 121.43,
123.18, 125.86, 127.22, 124, 121.17, 123.21, 122.17, 122.06,
121, 121.94, 123.6, 125.71, 128.72, 127.93, 124.85, 126.46,
125.56, 126.71, 129.72, 130.71, 130, 132.22, 136.01, 135.53,
136.39, 136.99, 137.88, 136.96, 137.42, 137.4, 135.77, 136.31,
135.38, 136.74, 141.99, 144.3, 144.3, 145.09, 139.85, 139.67,
132.49, 128.71, 130.22, 131, 131.45, 129.69, 130.17, 132.63,
131.63, 131.05, 131.74, 133.61, 133.94, 134.21

]

microsoft = [

507.79, 506, 509.6, 511.09, 507.29, 504.98, 508.19, 510.73, 507.63, 511, 515.16, 522.82,
526.1, 525.95, 532.7,
530.98, 527.59, 524.66, 528.09, 531.7, 537.3, 538.25, 535.8, 555.45, 515.95, 517.62, 515,
518.29, 513.67, 506.79,
511.2, 512.09, 514.64, 513.37, 506.72, 508.3, 503.97, 505.03, 504.44, 506.78, 498.2, 498.75,
500.13, 493.5, 498.05,
500.76, 499.3, 498.04, 494.56, 491.85, 487.75, 483.46, 481, 478.74, 480.69, 479.18, 480.42,
475.47, 472.8, 473.43,
473.34, 469.65, 465.69, 464.14, 462.11, 461.68, 461.72, 462.52, 460.95, 453.69, 460.25,
457.78, 458.34, 459.59,
454.36, 456.19, 453.9, 450.67, 449.37, 440.74, 443.67, 438.12, 437.73, 439.5, 439.44, 436.99,

```

396.66, 395.1, 392.74,
392.16, 388.45, 380.39, 367.77, 364.48, 374.32, 381.61, 391.89, 394.65, 390.05, 383.9,
393.23, 373.65, 371, 374.59,
377.48, 385.08, 382.85, 377.07, 389.13, 392.24, 395.31, 396.36, 395.4, 391.74, 391.79,
389.68, 387.37, 392.71,
390.23, 385.32, 385.22, 386, 386.4, 394.8, 402.15, 401.67, 392.58, 398.82, 397.63, 405.74,
403.6, 401.92, 409.37,
418.05, 419.31, 415.49, 410.6, 408.91, 411, 410.75, 412.49, 415.46, 418.65, 418.2, 413.83,
413.92, 415.41, 420.69,
422.86, 446.88, 448.38, 435.2, 446.65, 446.75, 447.27, 430.9, 434.48, 429.49, 428.15, 419.74,
418.5, 424.71, 426.97,
430.65, 434.32, 424.03, 426.07, 426.73, 427.55, 435.22, 440.94, 439.6, 437.65, 443.74,
443.18, 452.65, 455.29,
452.18, 451.43, 456.16, 450.35, 449.62, 448.33, 446.1, 444.66, 439.67, 432.47, 433, 424.88,
427.23, 429.04, 421.08,
417.4, 419.78, 417.29, 417.94, 418.4, 422.8, 428.17, 429.33, 424.44, 424.81, 426.5, 426.85,
420.45, 414.9, 410.42,
]

```

```

days = np.array(range(len(google)))

```

```

# Нормализуем данные
scaler = MinMaxScaler()
google_norm = scaler.fit_transform(np.array(google).reshape(-1, 1)).flatten()
apple_norm = scaler.fit_transform(np.array(apple).reshape(-1, 1)).flatten()
microsoft_norm = scaler.fit_transform(np.array(microsoft).reshape(-1, 1)).flatten()

```

```

fig, ax = plt.subplots(figsize=(12, 6))

```

```

ax.plot(days, google_norm, label='Google', linewidth=1)
ax.plot(days, apple_norm, label='Apple', linewidth=1)
ax.plot(days, microsoft_norm, label='Microsoft', linewidth=1)

```

```

ax.set_xlabel("День")
ax.set_ylabel("Нормализованная цена")
ax.set_title("Динамика акций (2021)")
ax.grid(True, alpha=0.3)
ax.legend()

```

```

plt.tight_layout()

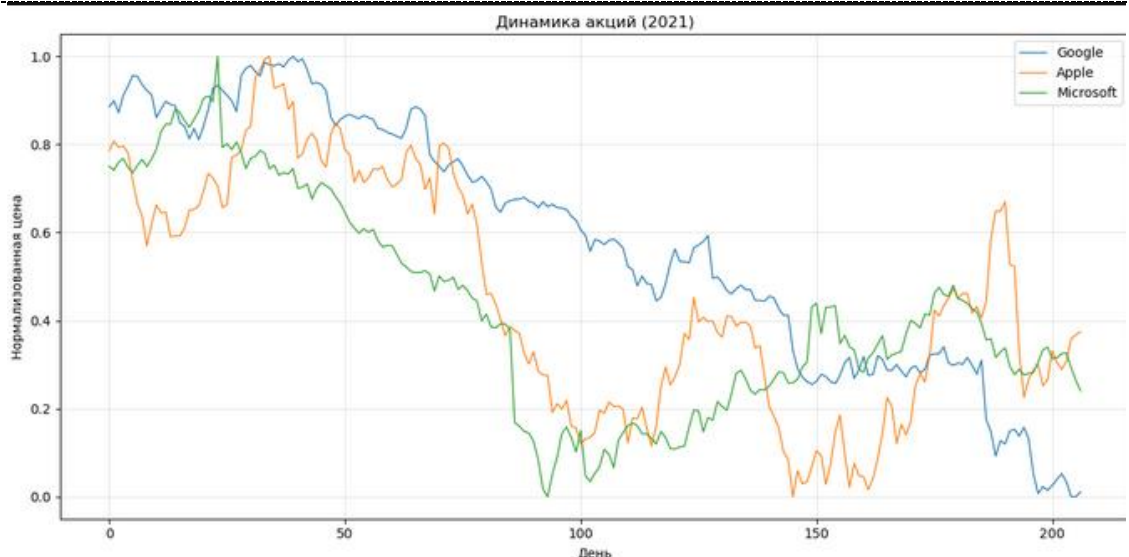
```

```

plt.show()

```

Ответ:



4.3.5 Задание

1.	Задача: Создайте простейший калькулятор, включающий основные действия для двух переменных '+', '-', '·', '÷', а также вычисление следующих функций: e^{x+y}, $\sin(x+y)$, $\cos(x+y)$, x^y. Решение: <pre># 4.3.5 def calc(x,y,operation): match (operation): case '+': return x + y case '-': return x-y case '*': return x*y case '/': return x/y case 'e': return np.exp(x+y) case 'sin': return np.sin(x+y) case 'cos': return np.cos(x+y) case '^': return np.pow(x,y) def main(): while True: x = int(input("Введите первое число: ")) y = int(input("Введите второе число: ")) operation = input("Выберите операцию: ") if operation == '0': break else: res = calc(x,y,operation) print(res) main()</pre> Ответ: Введите первое число: 15 Введите второе число: 17 Выберите операцию: cos 0.8342233605065102 Введите первое число: 0 Введите второе число: 0 Выберите операцию: 0
----	--